Lexical Analysis using Lex Tool

Program (lexer.l)

```
%{
#include <stdio.h>
#include <string.h>

int keywords = 0, identifiers = 0, numbers = 0, operators = 0, punctuation = 0;
%}

%%

"int"|"float"|"char"|"double"|"void"|"if"|"else"|"while"|"for"|"return"|"switch"|"case"|"break"|"continue" {
    printf("KEYWORD      : %s\n", yytext); keywords++;
}

[a-zA-Z_][a-zA-Z0-9_]* {
    printf("IDENTIFIER   : %s\n", yytext); identifiers++;
}

[0-9]+(\.[0-9]+)? {
    printf("NUMBER       : %s\n", yytext); numbers++;
}

"+"|"-"|"*"|"/"|"%"|"="|"=="|"!="|"<"|">"|"<="|">="|"&&"|"||"|"!" {
    printf("OPERATOR     : %s\n", yytext); operators++;
}

[(){}\[\];,] {
    printf("PUNCTUATION  : %s\n", yytext); punctuation++;
}

[ \t\n]+    ;   /* ignore whitespace */

. {
    printf("UNKNOWN      : %s\n", yytext);
}

%%

int yywrap() { return 1; }

int main() {
    printf("Enter input (Ctrl+D to end):\n");
    yylex();
    printf("\n--- Summary ---\n");
    printf("Keywords    : %d\n", keywords);
    printf("Identifiers : %d\n", identifiers);
    printf("Numbers     : %d\n", numbers);
    printf("Operators   : %d\n", operators);
    printf("Punctuation : %d\n", punctuation);
    return 0;
}
```

How to run

```
lex lexer.l
gcc lex.yy.c -o lexer
./lexer
```

Sample input

```
int a = 10 + b;
```

Sample output

```
KEYWORD      : int
IDENTIFIER   : a
OPERATOR     : =
NUMBER       : 10
OPERATOR     : +
IDENTIFIER   : b
PUNCTUATION  : ;
```